
画像判別（2）

1 はじめに

手書き文字認識データセットであるMNISTデータセットを用いて，画像を説明変数として手書き文字認識を行う方法を実習した．そこではScikite Learnを用いたが，今回は自分でニューラルネットワークを定義する方法を実習する．これは，将来的に，畳み込みニューラルネットワーク (Convolutional Neural Network, CNN) を用いる方法について実習する前提として，自分でネットワーク構造を定義する方法について学ぶため

ある.

2 画像を説明変数とする方法

画像を説明変数とする方法には以下のような例がある．

- 画像のビット単位の情報を説明変数とする．
- CNNを用いて画像から抽出した特徴を説明変数とする．
- CNNに入力するする前に，事前に画素の輝度差，コントラスト，形状情報等を数値データ化して利用する．
- 画像データとテキスト（メタデータ）や数値

データを同時に説明変数として利用する．（マルチモーダル学習）

このうち古くから行われていたのが，先日実習した1の方法である．今回は2について実習する．

3 画像を説明変数とする方法

1の方法では，画素ごとに数値化して，それを左上から右下の画素に向けて一列に並べるようにして説明変数とする．このため，2次元空間における上下左右の関係が壊れてしまう．

2の方法では，2次元空間における上下左右の関係を保持したまま，空間における画像の特徴量を扱うことができる．ただ，それがどのような特徴量であるかわかりにくい（説明しにくい）ことがある．

4 MLPの復習

前回の講義においてはScikit-learnを用いたので、ネットワーク構造を自分で構成することができなかった。最初にこの点を学習していくこととする。

そのために、今回はpytorchを利用する。あわせて、MNISTのデータについてもpytorchを利用することから説明を始める。

5 ライブラリの準備

CNNを利用する場合，KERAS.tensorflowやpytorchを利用する．ここではpytorchを利用することとする．

環境を整備するためにインストールのであるが，

1 `https://pytorch.org/`

へアクセスするとインストール方法が示される．
例えば以下のようなになる．

1 `pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu126`

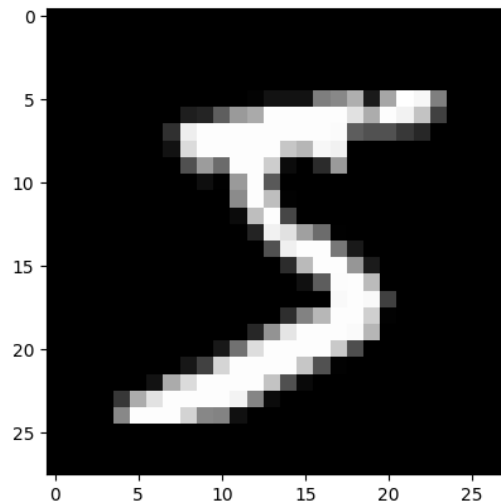
6 MNIST とは(1)

- 手書き数字を認識するために用いられる画像データセット
- 手書き数字のグレースケール画像(image)データと，数字を表すラベル(label)データからなる．
- 画像データは28行28列のグレースケールの数値を1次元に配列に格納している．
- scikit-learn や keras などのデータセットから利用できる．

7 MNIST とは(2)

```
1 import torch
2 import torchvision
3 import torchvision.transforms as transforms
4 import matplotlib.pyplot as plt
5
6 train_data = torchvision.datasets.MNIST(root='./data',
7                                         train=True,
8                                         transform=transforms.
9                                             ToTensor(),
10                                         download = True)
11 train_loader = torch.utils.data.DataLoader(dataset=train_data,
12                                             batch_size=256,
13                                             shuffle=True)
14
15 test_data = torchvision.datasets.MNIST(root='./data',
16                                         train=False,
17                                         transform=transforms.
18                                             ToTensor(),
19                                         download = True)
20 test_loader = torch.utils.data.DataLoader(dataset=test_data,
```

```
19                                     batch_size=256,  
20                                     shuffle=True)  
21  
22 fig, label = train_dataset[0]  
23 print("fig : {}, label : {}".format(fig,label))  
24 mpl.imshow(fig.view(-1,28), cmap=mpl.cm.gray)  
25 mpl.show()
```



8 プログラム例の説明

- 1行目から4行目：Pytorchのライブラリをインポートする．
- 6行目から12行目：学習データを導入する．
- 14行目から20行目：テストデータを導入する．
- 22行目から25行目：MNISTの最初の画像データを表示する．

9 ニューロンモデル

$$y = g\left(\sum_{i=1}^n w_i x_i - \theta\right) \quad (1)$$

- 入力: $x = \{x_1, x_2, \dots, x_n\}$
- 出力: y
- 重み係数 $w = \{w_1, w_2, \dots, w_n\}$
- 閾値 θ
- 活性化関数 (伝達関数) $g()$

閾値 θ を超えたときに，活性化関数が出力する．

10 活性化関数 (1)

- ステップ関数

$$g(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases} \quad (2)$$

- 対称型関数

$$g(x) = \tanh(x) \quad (3)$$

11 活性化関数 (2)

- 線形関数

$$g(x) = x \quad (4)$$

- ReLU 関数

$$g(x) = \begin{cases} x & 0 < x \\ 0 & x \leq 0 \end{cases} \quad (5)$$

12 活性化関数 (3)

- シグモイド関数

$$g(x) = \frac{1}{1 + e^{-x}} \quad (6)$$

- Softmax関数 ($k = 2$ においてシグモイド関数に一致する)

$$g(x) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}} \quad (7)$$

13 誤差逆伝播

閾値や重み係数を決定するために誤差逆伝播 (Back Propagation method) を用いる。

1. 入力 x と出力 y の正しい組み合わせ（教師信号）を多数用意する。
2. 重み w と閾値 θ をランダムに与える。
3. 入力 x を与えて出力 y' を得る。
4. y' と教師信号 y の差を最小にするようにパラメータを更新する。
5. 3,4 を繰り返す。

14 損失関数

- y' と教師信号 y の差を評価するために損失関数 (Loss Function) を用いる.
- 判別問題や回帰問題など, 問題に応じた異なる損失関数を用いる.
- 判別問題では交差エントロピー関数を用いることが多い.
- 回帰問題では平均二乗誤差 (Mean Squared Error, MSE): 等を用いる.

15 最適化手法

閾値や重み係数を決定するために最適化手法が用いられる．代表的な方法に以下の方法がある．

- 最急降下法 (Gradient Descent Method, GDM)
- Adaptive Gradient algorithm (AdaGrad)
- AdaDelta algorithm
- RMSprop (RMS) algorithm
- Momentum algorithm
- Adaptive Moment Estimation (Adam) algorithm

16 MLPによってMNISTを判別 するプログラム例

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import torchvision
5 import torchvision.transforms as transforms
6 import matplotlib.pyplot as plt
7
8 class NNet(nn.Module):
9     def __init__(self, in_size, h1_size, out_size):
10         super().__init__()
11         self.l1 = nn.Linear(in_size, h1_size)
12         self.l2 = nn.Linear(h1_size, out_size)
13         self.act = nn.Sigmoid()
14
15     def forward(self, x):
16         h1 = self.act(self.l1(x))
17         y = self.l2(h1)
```

```
18         return y
19
20
21 train_data = torchvision.datasets.MNIST(root='./data',
22                                         train=True,
23                                         transform=transforms.ToTensor(),
24                                         download = True)
25 train_loader = torch.utils.data.DataLoader(dataset=train_data,
26                                             batch_size=256,
27                                             shuffle=True)
28
29 test_data = torchvision.datasets.MNIST(root='./data',
30                                         train=False,
31                                         transform=transforms.ToTensor(),
32                                         download = True)
33 test_loader = torch.utils.data.DataLoader(dataset=test_data,
34                                           batch_size=256,
35                                           shuffle=True)
36
37 in_size = 28*28
38 h1_size = 1024
39 out_size = 10
```

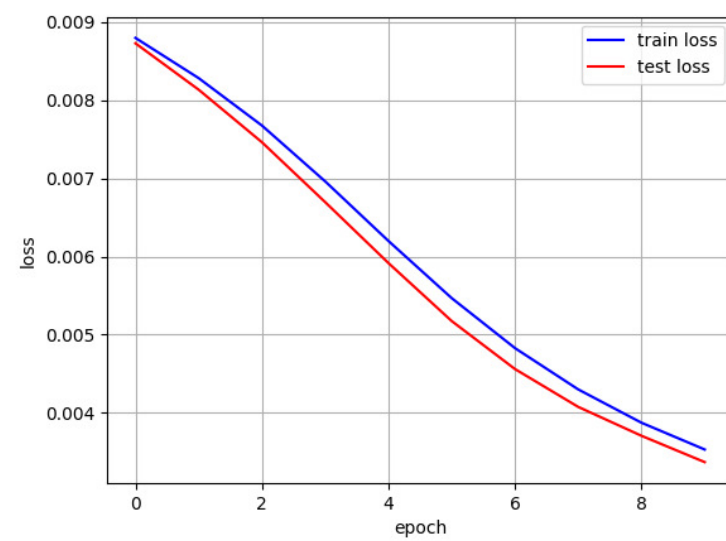
```
40 device = 'cpu' # 'cuda' if torch.cuda.is_available() else 'cpu'
41 model = NNet(in_size, h1_size, out_size).to(device)
42 print(model)
43
44 # 損失関数：交差エントロピー誤差関数
45 criterion = nn.CrossEntropyLoss()
46
47 # 最適化法：SGD（確率的勾配降下法）
48 optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
49
50
51 def train_model(model, train_loader, criterion, optimizer, device):
52     train_loss = 0.0
53     num_train = 0
54     model.train()    #学習モード
55
56     for i, (images, labels) in enumerate(train_loader):
57         num_train += len(labels)
58
59         images, labels = images.view(-1, 28*28).to(device), labels.
60             to(device)
```

```
61         optimizer.zero_grad()
62         outputs = model(images)
63         loss = criterion(outputs, labels)
64         loss.backward()
65         optimizer.step()
66         train_loss += loss.item()
67
68     train_loss = train_loss / num_train
69
70     return train_loss
71
72
73 def test_model(model, test_loader, criterion, device):
74     test_loss = 0.0
75     num_test = 0
76     model.eval()          #評価モード
77
78     with torch.no_grad():
79         for i, (images, labels) in enumerate(test_loader):
80             num_test += len(labels)
81             images, labels = images.view(-1, 28*28).to(device),
                labels.to(device)
```

```
82         outputs = model(images)
83         loss = criterion(outputs, labels)
84         test_loss += loss.item()
85     test_loss = test_loss / num_test
86     return test_loss
87
88
89 def lerning(model, train_loader, test_loader, criterion, optimizer,
90            num_epochs, device):
91     train_loss_list = []
92     test_loss_list = []
93
94     for epoch in range(1, num_epochs+1, 1):
95         train_loss = train_model(model, train_loader, criterion,
96                                optimizer, device)
97         test_loss = test_model(model, test_loader, criterion,
98                               device)
99         print("{}    train_loss : {:.5f}    test_loss : {:.5f}" .
100               format(epoch, train_loss, test_loss))
101         train_loss_list.append(train_loss)
102         test_loss_list.append(test_loss)
103     return train_loss_list, test_loss_list
```



```
100
101
102 num_epochs = 10
103 train_loss_list, test_loss_list = lerning(model, train_loader,
      test_loader, criterion, optimizer, num_epochs, device=device)
104
105 mpl.plot(range(len(train_loss_list)), train_loss_list, c='b', label
      ='train loss')
106 mpl.plot(range(len(test_loss_list)), test_loss_list, c='r', label='
      test loss')
107 mpl.xlabel("epoch")
108 mpl.ylabel("loss")
109 mpl.legend()
110 mpl.grid()
111 mpl.show()
```



17 プログラム例の説明

- 8行目から18行目：ネットワークの定義．今回は中間層は1層となっている．活性化関数はシグモイド関数 `Sigmoid()` である．入力変数は x ，出力変数は y ，中間変数は $h1$ となっている．
- 37行目から42行目：ネットワークパラメータを定義し，表示する．
- 44行目から48行目：損失関数と最適化法を定義する．
- 51行目から70行目：モデル学習の定義

- 73行目から86行目：モデルテストの定義
- 89行目から99行目：学習プロセスの定義
- 102行目から111行目：学習し，結果を出力する．

プログラムにおいて，8行目から18行目の定義を変更すればネットワークを変更できる．また，44行目から48行目の損失関数と最適化法の定義も適時変更する必要がある．

18 演習問題

- 中間層のノード数をそれぞれ1/2,2倍として実験して精度や計算時間を比較しなさい.
- ネットワークの中間層を2層,3層と増やしてみなさい.
- その場合において, 中間層のノード数をそれぞれ1/2,2倍として実験して精度や計算時間を比較しなさい.